

BPhO Computational Challenge Task 6 Electron Diffraction

Abhinav Nayak

May 2026

1 Abstract

The electron diffraction experiment was first conducted by Clinton Davisson and Lester Germer in 1927 [1]. It is significant because it demonstrates the wave nature of electron, confirming de Broglie's hypothesis, which was instrumental in the development of quantum mechanics [2].

In this paper, we create a computer model which plots the electron diffraction pattern produced on the basis of the accelerating potential difference of the cathode. Then we calculate the spacing by plotting the maxima points on a graph of $\frac{1}{\sqrt{V}}$ against $\sin \frac{\phi}{2}$, and calculating the gradient.

2 Creating the Computer Model

We simulated the electron diffraction experiment, producing the expected interference pattern based on the accelerating potential difference.

2.1 Calculating Wavelength

Our first step is to calculate the wavelength of an electron on the basis of the input voltage. We can do this by starting with the de Broglie wavelength equation, $\lambda = h/p$, and the equation for momentum, $p = mv$.

When the electron is accelerated through a p.d. of V , all the electrical energy is transferred to the kinetic energy store of the electron. This can be used to calculate the momentum of the electron.

$$eV = \frac{1}{2}mv^2$$

We can substitute the equation for momentum (rearranged for velocity) into the equation:

$$eV = \frac{1}{2}mv^2$$

$$\lambda = \frac{h}{\sqrt{2meV}} \quad (1)$$

We can now substitute this into the de Broglie wavelength equation [3]:

$$p = \sqrt{2meV} \quad (2)$$

This is our code for this section:

```
fprintf("V      --->    ( 1-5kV )\n")
V = input("Enter voltage : ");
V = V*1000;
lambda = h/(sqrt(2*m*e*V));
```

2.2 Calculating Interference

Our next step is to use the wavelength to calculate the brightness across the phosphor screen. Initially, we considered creating a vector of evenly spaced points for phi (the angle at which the electron is diffracted) and then calculating the corresponding x (radius of the ring). We decided against this because the rings would be more concentrated towards the edges of the computer model due to the use of sine in our equation. Therefore, we instead created a vector of evenly spaced points for x, then calculated the corresponding value of phi for each x.

```
x = linspace(0,65e-3,rings);
phi = 0.5.*asin(x./r);
```

Next, we needed to calculate the brightness for each radius x ring. To do this we needed to calculate the phase difference between the electrons using Bragg's Law [3].

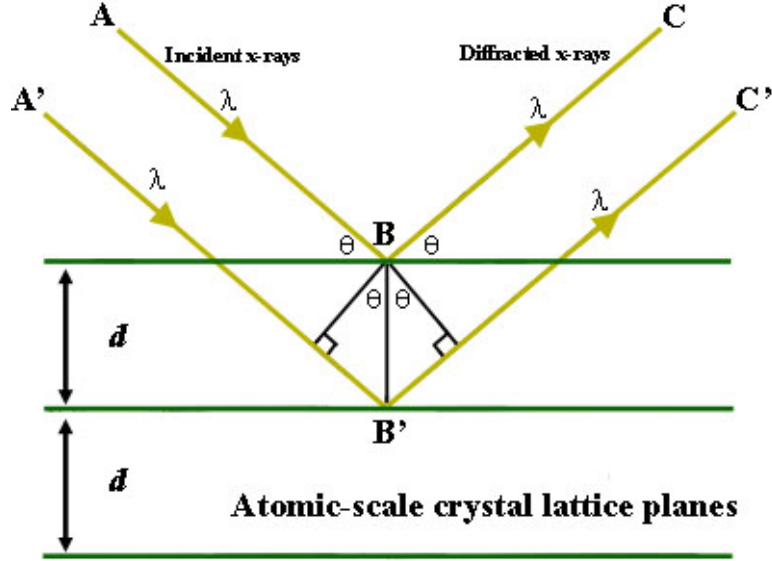


Figure 1: Diagram illustrating Bragg's Law [4]

$$\text{Path Difference} = 2d \sin \frac{\phi}{2} \quad (3)$$

$$\text{Phase Difference} = ((2d \sin \frac{\phi}{2}) \bmod 1) 2\pi$$

The brightness needs to be high for a phase difference close to 0 or 2π and low for a phase difference of π . The cosine function is ideal to calculate this. Taking the cosine of the phase difference, adding 1 and dividing by two ensures all values are between 0 and 1.

$$\text{Brightness} = \frac{\cos(((2d \sin \frac{\phi}{2}) \bmod 1) 2\pi)}{2} \quad (4)$$

We use this function to calculate the brightness:

```
function out = brightness(d)
    global lambda phi
    path_diff = (2*d.*sin(phi./2))/lambda;
    phase_diff = rem(path_diff,1).*2*pi;
    out = (1+cos(phase_diff))./8;
end
```

(The last step is divided by 8 instead of two because the brightnesses will be added, and the final plotted brightness needs to be between 0.5 and 1 to get

a suitable shade of green, so the range for the outputted brightness used here is 0 to 0.25.)

This is applied to both distances:

```
d1 = brightness(0.123e-9);
d2 = brightness(0.213e-9);
total_brightness = (d1+d2);
```

2.3 Plotting Interference Pattern

We then plot rings for each value of x , which are plotted as parametric equations. The brightness is controlled using the RGB triplet, with red and blue set at 0 and green being a value between 0.5 and 1 based on the calculated brightness. A value of 0.5 plots dark green, which represents the neutral colour of the phosphor screen, where destructive interference occurs. A value of 1 displays in light green, which represents constructive interference.

```
theta = linspace(0, 2*pi, 100);
hold on
for n = 1:rings
    radius = x(n);
    x_circle = radius*cos(theta);
    y_circle = radius*sin(theta);
    green = 0.5+total_brightness(n);
    plot(x_circle,y_circle,'Color',[0,green,0])
end
```

3 Checking the Computer Model

To verify the computer model, we emulated a real experiment by running the computer model 10 times with different accelerating potential differences and plotting a graph of $\frac{1}{\sqrt{V}}$ against $\sin \frac{\phi}{2}$.

3.1 Storing the Maxima

First, we created two matrices of zeros (one for each spacing) which would be used to store the local maxima. Each row contains all the maximum points for one value of V . We chose to do this because the points are stored by looping through the different accelerating potential differences one-by-one. However, when plotting, we will need to plot the n th ring for each separate accelerating potential difference.

```
points1 = zeros(10, 10);
points2 = zeros(10, 10);
```

Next, we run the computer model, as in section 2. We search the brightness curve for the local maxima and store it in our matrices.

```

d1 = brightness(0.123e-9);
previous = 0;
peak = 0;
n = 1;
for j = 1:rings
    current = d1(j);
    if current < previous && peak == 1
        points1(i,n) = sin(phi(j-1)/2);
        previous = current;
        peak = 0;
        n = n+1;
    elseif current > previous
        peak = 1;
    end
    previous = current;
end

```

This is repeated for the second spacing and repeated for all the accelerating potential differences.

	1	2	3	4	5	6	7	8	9	10
1	0	0.1578	0.3157	0	0	0	0	0	0	0
2	0	0.1314	0.2627	0	0	0	0	0	0	0
3	0	0.1148	0.2297	0.3445	0	0	0	0	0	0
4	0	0.1033	0.2067	0.3101	0	0	0	0	0	0
5	0	0.0947	0.1894	0.2842	0.3781	0	0	0	0	0
6	0	0.0879	0.1759	0.2638	0.3518	0	0	0	0	0
7	0	0.0824	0.1649	0.2473	0.3297	0	0	0	0	0
8	0	0.0778	0.1557	0.2336	0.3115	0	0	0	0	0
9	0	0.0740	0.1479	0.2219	0.2959	0.3696	0	0	0	0
10	0	0.0706	0.1412	0.2117	0.2824	0.3529	0	0	0	0

Figure 2: An example output for the "points" matrix.

3.2 Plotting the graph

We deal with each diffraction order separately by extracting each column separately.

```

list_distance1 = [];
for column=2:10
    list = points1(:,column); % Extracts column from matrix
    nonzero = [];
    for index = 1:length(list)
        if list(index) ~= 0
            nonzero(end+1) = index;
        end
    end
end

```

Each diffraction order is plotted with a different colour, and each spacing is plotted with a different line style.

```
colours = {[1 0 0],[0 1 0],[0 0 1],[1 1 0],[0 1 1],[1 0 1],[0 0 0],
[1 0.5 0],[0.5 0 0.5],[1 0.4 0.7]};
plot(list(nonzero),list_root_V(nonzero),'-','Color',colours{column})
```

3.3 Calculating the Spacing

Using the polyfit function, we estimate the gradient of the lines produced.

```
gradient = polyfit(list(nonzero),list_root_V(nonzero),1);
```

Using equations (1) and (3), we can rearrange to find what the gradient represents:

$$2d \sin \theta = n\lambda$$

$$2d \sin \frac{\phi}{2} = \frac{nh}{\sqrt{2meV}}$$

$$\frac{1}{V} = \frac{2d\sqrt{2me}}{nh} \sin \theta \quad (5)$$

Using this equation, we calculate the spacing.

```
list_distance1(end+1) = gradient(1)*((column-1)*h)/
(2*sqrt(2*m*e));
```

Before calculating the mean of all the values in the list of distances, we need to remove the values where the gradient is zero. This occurs when the gradient is calculated from just one point, when that diffraction order only exists in one accelerating voltage.

```
list_distance1 = list_distance1(list_distance1~=0)
list_distance2 = list_distance2(list_distance2~=0)
distance1 = mean(list_distance1)
distance2 = mean(list_distance2)
```

4 Conclusion

Using this process, we obtain the following values for the spacing of graphite.

```
distance1 =
1.2336e-10
```

```
distance2 =
2.1319e-10
```

This is close to the actual values of the graphite spacing, with a 0.29% and 0.09% uncertainty respectively.

References

- [1] Wikipedia contributors, “Davisson–germer experiment.” https://en.wikipedia.org/wiki/Davisson%E2%80%93Germer_experiment, 2026. Accessed: 24 July 2026.
- [2] R. Nave, “Davisson-germer experiment.” <http://hyperphysics.phy-astr.gsu.edu/hbase/quantum/DavGer2.html>. HyperPhysics, Georgia State University. Accessed: 24 July 2026.
- [3] Dr. Andrew French, “Bpho computational challenge 2026 quantum mechanics.” https://eclecticon.info/physics_bpho_compphys.htm, 2026. Accessed: 24 July 2026.
- [4] Science Education Resource Center, “Bragg’s law.” https://serc.carleton.edu/research_education/geochemsheets/BraggsLaw.html. Carleton College. Accessed: 24 July 2026.